

03

Estrutura Condicional em Python

Handout de Estudos · if · elif · else
Switch Case · Match-case · break /
continue

MÓDULO

Introdução à Programação
Estruturada em Python

Professores: Luciano · Rafael · Turatti

Turma: 2º Semestre · CC e SI

Material: Handout complementar à aula expositiva

Como usar: Leia na ordem, execute os exemplos no Python e
resolva os exercícios ao final

SUMÁRIO · 4 CAPÍTULOS + EXERCÍCIOS

01	Fundamentos das Estruturas Condicionais	p. 2
02	A família if / elif / else	p. 4
03	Switch Case e o Match-case do Python	p. 8
04	Break e Continue — controle de loops	p. 10
★	Lista de Exercícios (8 questões)	p. 12

Este handout é o material oficial de estudo do Módulo 3. Ele foi construído para acompanhar a aula expositiva e serve como referência rápida para revisão e resolução dos exercícios. **Todos os exemplos de código podem ser copiados e executados diretamente no Python 3.10 ou superior.**

Antes de começar, garanta que você tem o Python instalado (versão 3.10+ para testar o match-case). Abra o terminal e digite `python --version`. Se aparecer `Python 3.10.x` ou superior, você está pronto.

— CAPÍTULO 01 · FUNDAMENTOS

O que são estruturas condicionais

Estruturas condicionais são comandos que permitem ao programa **executar blocos diferentes de código** dependendo de uma expressão avaliada como verdadeira (`True`) ou falsa (`False`). Elas são o mecanismo fundamental pelo qual um programa deixa de ser uma sequência rígida de instruções e passa a **tomar decisões** com base no estado dos dados e na entrada do usuário.

Em Python, toda condição é avaliada como um valor booleano. O interpretador pergunta "isso é verdadeiro?" e, conforme a resposta, escolhe qual bloco executar — podendo pular, repetir ou desviar o fluxo. Esse conceito simples sustenta praticamente toda a lógica de qualquer programa não-trivial: validar entradas, escolher caminhos em menus, classificar dados, controlar jogos, decidir quem pode acessar um sistema e assim por diante.

Antes de escrever um único `if`, porém, precisamos dominar o vocabulário que sustenta as condições: os **operadores relacionais** (que comparam valores) e os **operadores lógicos** (que combinam condições). A tabela abaixo resume os operadores que usaremos ao longo de todo o módulo.

1.1 Operadores relacionais e lógicos

OPERADOR	SIGNIFICADO	EXEMPLO	RESULTADO
=	Igual a	5 = 5	True
≠	Diferente de	5 ≠ 3	True
>	Maior que	7 > 4	True
<	Menor que	2 < 8	True
≥	Maior ou igual	5 ≥ 5	True
≤	Menor ou igual	4 ≤ 3	False
and	E lógico (ambos)	True and False	False
or	Ou lógico (ao menos um)	True or False	True
not	Negação (inverte)	not True	False

Combine operadores com parênteses para evitar ambiguidade: `(idade ≥ 18) and (tem_cnh)` é mais legível e seguro do que `idade ≥ 18 and tem_cnh`. A precedência do Python é `not > and > or` — quando em dúvida, use parênteses.

```
idade = 20
tem_cnh = True
if (idade ≥ 18) and tem_cnh:
    print("Pode dirigir")
```

PYTHON

1.2 Fluxograma — o caminho da decisão

Um fluxograma representa graficamente o caminho que o programa percorre. O **losango** indica um ponto de decisão (uma condição); as setas mostram os dois caminhos possíveis — verdadeiro (sim) e falso (não). Cada caminho leva a um bloco de ação diferente. Antes de escrever um `if` complexo, desenhe mentalmente o fluxograma: isso evita bugs de lógica.

```
nota = 8

if nota ≥ 7:
    print("Aprovado")      # caminho do SIM
else:
    print("Reprovado")     # caminho do NÃO
```

EXEMPLO

Se `nota = 8`, a condição `nota ≥ 7` é verdadeira e o programa imprime "Aprovado". Se `nota = 5`, a condição é falsa e o programa pula para o `else`, imprimindo "Reprovado". Esse é o padrão mais simples de decisão binária, que estudaremos em detalhe no próximo capítulo.

A família das condicionais do Python

A família `if` / `elif` / `else` é a estrutura condicional mais utilizada em Python. Aprenderemos a sintaxe, as três variações (simples, binária e em cadeia) e as armadilhas que mais derrubam iniciantes. Ao final deste capítulo, você será capaz de escrever qualquer lógica condicional clássica em Python.

2.1 if simples — executa ou pula

O `if` mais básico avalia uma condição. Se for verdadeira, executa o bloco indentado. Se for falsa, o programa simplesmente pula o bloco e segue. **O bloco pertence ao `if` enquanto estiver indentado** — qualquer linha alinhada com o `if` (sem recuo) está fora da condicional.

EXEMPLO · VERIFICAÇÃO DE IDADE

```
idade = int(input("Sua idade: "))
if idade ≥ 18:
    print("Maior de idade")
    print("Pode tirar CNH")
print("Fim do programa")  # sempre executa
```

Sintaxe obrigatória: palavra-chave `if`, espaço, condição, dois pontos (`:`) — sem isso, dá `SyntaxError`. Em seguida, enter e **indentação de 4 espaços** (padrão PEP 8). O Python usa indentação em vez de chaves como C/Java — isso força código legível mas exige disciplina.

2.2 if / else — decisão binária

Quando há exatamente dois caminhos possíveis, usamos `if` acompanhado de `else`. Se a condição for verdadeira, executa o bloco do `if`; caso contrário, executa o bloco do `else`. Não há como escapar: um dos dois sempre roda.

EXEMPLO · PAR OU ÍMPAR

```
numero = int(input("Digite um inteiro: "))
if numero % 2 == 0:
    print(f"{numero} é PAR")
else:
    print(f"{numero} é ÍMPAR")
```

SAÍDA PARA ENTRADA = 8
> 8 é PAR

SAÍDA PARA ENTRADA = 7
> 7 é ÍMPAR

O operador `%` (módulo) devolve o resto da divisão e é a forma clássica de verificar paridade. Para `numero = 8`: `8 % 2 = 0`, condição verdadeira, imprime PAR. Para `numero = 7`: `7 % 2 = 1`, condição falsa, vai para o `else`, imprime ÍMPAR. O `else` é opcional — podemos ter `if` sem `else`, mas não `else` sem `if`.

2.3 if / elif / else — múltiplos caminhos

Quando há três ou mais caminhos, encadeamos com `elif` (contração de *else if*). O Python avalia as condições de cima para baixo e executa **apenas o primeiro bloco** cuja condição seja verdadeira. As demais não são testadas — é o chamado curto-circuito.

EXEMPLO · CLASSIFICAÇÃO DO IMC

```
imc = 22.5

if imc < 18.5:
    print("Abaixo do peso")
elif imc < 25:
    print("Peso normal")
elif imc < 30:
    print("Sobrepeso")
else:
    print("Obesidade")
```

VALOR DE IMC	SAÍDA
17	Abaixo do peso
22	Peso normal
27	Sobrepeso
32	Obesidade

⚠️ ATENÇÃO À ORDEM!

Coloque sempre as condições mais **específicas/restritivas** primeiro. No exemplo, `elif imc < 25` só é alcançado se `imc < 18.5` já foi falso — então ele implicitamente já inclui `imc ≥ 18.5`. Se invertêssemos para `if imc < 30` primeiro, todo mundo abaixo de 30 cairia ali e nunca veria as outras categorias.

2.4 Estudo de caso — situação do aluno

Programa que recebe duas notas, calcula a média e decide a situação acadêmica segundo o regulamento da instituição. Esse é um padrão que aparece em todos os sistemas acadêmicos — domine-o.

PROGRAMA COMPLETO

```
# Programa: situação do aluno
nota1 = float(input("Nota 1: "))
nota2 = float(input("Nota 2: "))
media = (nota1 + nota2) / 2

if media ≥ 7.0:
    situacao = "Aprovado"
elif media ≥ 5.0:
    situacao = "Recuperação"
else:
    situacao = "Reprovado"

print(f"Média: {media:.1f} – Situação: {situacao}")
```

NOTA 1	NOTA 2	MÉDIA	SAÍDA
8.0	9.0	8.5	Aprovado
6.0	7.0	6.5	Recuperação
4.0	3.0	3.5	Reprovado
7.0	7.0	7.0	Aprovado
5.0	5.0	5.0	Recuperação

2.5 Pegadinhas comuns do if

Quatro armadilhas responsáveis por mais de 80% dos bugs de iniciantes em condicionais. Memorize todas.

⚠ 1. = VS ==

`=` atribui, `==` compara. Em Python o erro é capturado em tempo de compilação (`SyntaxError`); em C/Java seria um bug silencioso e difícil de encontrar. **Crie o hábito desde já.**

⚠ 2. ESQUECER OS DOIS PONTOS

Os dois pontos (`:`) são obrigatórios ao final de `if`, `elif`, `else`, `for`, `while`, `def`, `class`. Esquecer = `SyntaxError`.

⚠ 3. INDENTAÇÃO INCONSISTENTE

Use sempre **4 espaços**. Nunca misture tabs e espaços no mesmo bloco. Configure o editor para "inserir espaços em vez de tabs" — isso evita 90% dos `IndentationError`.

⚠ 4. COMPARAR FLOATS COM ==

Floats têm imprecisão binária. `0.1 + 0.2 == 0.3` retorna `False` ! Compare com tolerância (epsilon) ou use `round()`: `abs(0.1 + 0.2 - 0.3) < 1e-9`.

— CAPÍTULO 03 • SWITCH & MATCH

Switch Case e o Match-case do Python

Outras linguagens têm `switch` desde os anos 70 (C, 1978). O Python propositalmente ficou sem por décadas — a filosofia era de que `if-elif-else` cobria os casos. Em 2021 (PEP 634), o Python 3.10 introduziu o `match-case`, que não é apenas um `switch` — é **pattern matching estrutural**, recurso emprestado de linguagens funcionais como Scala e Haskell.

3.1 Comparação entre linguagens

Para entender o que o Python fez, comparamos três formas de resolver o mesmo problema: dado um código de dia (1 a 7), imprimir o nome do dia. Veja como cada linguagem aborda o problema.

```
switch (dia) {
  case 1:
    print("Seg");
    break;      // obrigatório!
  case 2:
    print("Ter");
    break;      // obrigatório!
  default:
    print("Outro");
}
```

```
match dia:
  case 1:
    print("Seg")    # sem break
  case 2:
    print("Ter")    # sem break
  case _:
    print("Outro")
```

ASPECTO	SWITCH (C/JAVA/JS)	MATCH-CASE (PYTHON 3.10+)
Fall-through	Tem — precisa <code>break</code> em cada case	Não tem — cada case é exclusivo
Default	Palavra-chave <code>default</code>	Coringa <code>_</code> (underscore)
Poder	Só compara igualdade	Pattern matching estrutural
break	Obrigatório para evitar fall-through	Proibido — gera <code>SyntaxError</code>

3.2 Match-case em ação — roteamento de comandos

O `match-case` avalia uma expressão e tenta casá-la com cada padrão. O primeiro padrão que encaixar é executado; os demais são ignorados. O `_` é o coringa que casa com qualquer valor — equivalente ao `default` do `switch`. A verdadeira força do `match` aparece quando usamos **padrões estruturais**: ele desconstrói listas, tuplas e dicionários automaticamente.

```
comando = input("> ").strip().split()
```

EXEMPLO · ROTEAMENTO DE COMANDOS

```
match comando:
  case ["sair"]:
    print("Encerrando...")
  case ["ajuda"]:
    print("Comandos: sair, ajuda, olá <nome>")
  case ["olá", nome]:
    print(f"Olá, {nome}!")
  case ["somar", a, b]:
    print(int(a) + int(b))
  case _:
    print("Comando não reconhecido")
```

ENTRADA	PADRÃO CASADO	SAÍDA
"sair"	["sair"]	Encerrando...
"olá Maria"	["olá", nome] → nome="Maria"	Olá, Maria!
"somar 3 4"	["somar", a, b] → a="3", b="4"	7
"pular"	_ (coringa)	Comando não reconhecido

Veja o que acontece com entrada `"olá Maria"`: o `.split()` produz a lista `['olá', 'Maria']`. O `case ["olá", nome]` casa uma lista de exatamente 2 elementos onde o primeiro é `"olá"` — e captura o segundo na variável `nome`. Isso é **pattern matching estrutural**, recurso que aproxima o Python de linguagens funcionais como Scala e Haskell.

⚠️ DISPONIBILIDADE

`match-case` está disponível apenas a partir do **Python 3.10** (outubro de 2021). Verifique a versão com `python --version`. Se a empresa usa 3.9 ou anterior, `if-elif-else` ainda é a alternativa universal.

— CAPÍTULO 04 • BREAK / CONTINUE

Break e Continue — controle de loops

`break` e `continue` não são comandos soltos: na prática, **vivem dentro de um `if`**, que é o gatilho que decide QUANDO disparar a interrupção ou o salto. E o `break` obrigatório do `switch` tradicional (C/Java/JS) simplesmente desaparece no `match-case` do Python, porque cada `case` é exclusivo por padrão.

Regra fundamental: **`break` e `continue` são controles de LAÇO** (for, while). Isolados num `if` sem loop em volta, geram `SyntaxError`. E no `match-case` do Python, eles simplesmente não existem — `cases` são exclusivos por design.

4.1 break — interrompendo o laço

O `break` encerra imediatamente o laço (`for` ou `while`) mais interno. Quase nunca é usado solto — ele vive dentro de um `if`, que decide QUANDO o laço deve ser interrompido. O `if` é o gatilho; o loop é o alvo.

SINTAXE

```
for item in colecao:
    if condicao:           # if decide QUANDO
        break             # loop é encerrado
# fluxo continua aqui
```

EXEMPLO • BUSCA EM LISTA

```
numeros = [3, 7, 2, 9, 5, 1]
alvo = 9

for n in numeros:
    print(f"Verificando {n}...")
    if n == alvo:         # if dispara o break
        print(f"Achei {alvo}!")
        break
else:
    print(f"{alvo} não está na lista")
```

SAÍDA

```
> Verificando 3...
> Verificando 7...
> Verificando 2...
> Verificando 9...
> Achei 9!
```

Os números 5 e 1 nunca são visitados porque o `break` sai do `for` assim que encontra o 9. Note o `else` do `for`: é uma peculiaridade do Python — ele executa apenas quando o laço termina naturalmente (sem `break`). Útil para distinguir "terminei sem achar" de "achei e sai".

4.2 break no switch vs match-case

Aqui está a diferença crucial que confunde quem vem de C/Java/JS: nessas linguagens, o `break` dentro de `switch` é **obrigatório** para evitar o fall-through (continuar executando o próximo case). No `match-case` do Python, isso não existe — cada case é exclusivo por design, e tentar colocar `break` gera `SyntaxError`.

C / JAVA / JS — TRADICIONAL

```
switch (op) {
  case 1:
    fazA();
    break;    // obrigatório!
  case 2:
    fazB();
    break;    // obrigatório!
}
```

PYTHON 3.10+ — MODERNO

```
match op:
  case 1:
    fazA()      # sem break
  case 2:
    fazB()      # sem break
```

⚠ BREAK + MATCH = SYNTAXERROR

Em C/Java/JS, o `break` evita o fall-through. No `match-case` do Python, cada case é exclusivo por padrão — `break` é desnecessário e **gera `SyntaxError` se usado**.

4.3 continue — pulando iterações

O `continue` interrompe a iteração atual e salta para a próxima. Assim como o `break`, ele vive dentro de um `if` — que decide QUANDO pular. `continue` é exclusivo de loops (for/while): **NÃO existe `continue` em `if` isolado nem em `match-case`**.

EXEMPLO · SOMAR APENAS OS PARES

```
numeros = [1, 2, 3, 4, 5, 6, 7, 8]
soma = 0

for n in numeros:
    if n % 2 != 0:      # if filtra ímpares
        continue      # pula direto
    soma += n
    print(f"Somei {n}, parcial = {soma}")

print(f"Total: {soma}")
```

SAÍDA

```
> Somei 2, parcial = 2
> Somei 4, parcial = 6
> Somei 6, parcial = 12
> Somei 8, parcial = 20
> Total: 20
```

COMANDO	ONDE FUNCIONA	PAPEL DO IF
<code>break</code>	for / while (e switch em C)	Gatilho — decide QUANDO
<code>continue</code>	for / while apenas	Filtro — decide O QUE pular

Nenhum dos dois funciona em `match-case`. E nenhum dos dois funciona sem um loop em volta.

4.4 Estudo de caso — validação de senha

Programa que pede a senha até 3 vezes. Usa `continue` (dentro de `if`) para repetir entradas vazias, e `break` (dentro de `if`) para sair ao acertar. Observe como cada `break` / `continue` está sempre dentro de um `if` — esse é o padrão correto.

```
senha_correta = "python123"
tentativas = 0

while tentativas < 3:
    entrada = input("Senha: ")
    if entrada == "":           # if → gatilho
        print("Vazia, tente novamente")
        continue              # pula iteração
    tentativas += 1
    if entrada == senha_correta: # if → gatilho
        print("Acesso concedido!")
        break                 # sai do while
    print("Senha incorreta")

if tentativas == 3:
    print("Conta bloqueada")
```

PROGRAMA COMPLETO

Padrão correto: cada `break` / `continue` está dentro de um `if`. O `if` decide a CONDIÇÃO; o loop sofre a AÇÃO. Nunca use `break` / `continue` solto.

4.5 Pegadinhas comuns de break/continue

⚠ 1. BREAK/CONTINUE PRECISAM DE LOOP

Fora de `for` / `while`, "`if cond: break`" gera `SyntaxError`. `break` e `continue` são controles de LAÇO, não de `if` isolado.

⚠ 2. BREAK NO MATCH-CASE É PROIBIDO

Em Python, `match-case` não usa `break` — cada case é exclusivo por padrão. Colocar `break` gera `SyntaxError`.

⚠ 3. BREAK SÓ SAI DO LAÇO INTERNO

Em loops aninhados, `break` encerra apenas o `for` / `while` mais interno. O externo continua normalmente. Para sair de vários níveis, use exceções ou refatore com funções e `return`.

⚠ 4. CONTINUE NO WHILE VIRA LOOP INFINITO

Se o incremento estiver DEPOIS do `continue`, o contador nunca avança. Sempre incremente ANTES do `continue`, ou use `for` (que incrementa sozinho).

★ LISTA DE EXERCÍCIOS

8 questões para fixação

Resolva os exercícios a seguir para fixar o conteúdo. **Tente antes de olhar qualquer referência** — só consulte o material quando travar de verdade. Cada questão indica o capítulo relacionado e o nível de dificuldade.

Q1 MAIOR DE DOIS NÚMEROS

FÁCIL

Capítulo 2. Escreva um programa que receba dois números inteiros do usuário e imprima qual é o maior. Considere o caso de empate (imprima "São iguais"). Use apenas `if` / `elif` / `else`.

Q2 CLASSIFICAÇÃO DE TRIÂNGULO

MÉDIO

Capítulo 2. Receba três lados de um triângulo. Primeiro verifique se formam um triângulo válido (a soma de dois lados deve ser maior que o terceiro, para todos os pares). Se válido, classifique em **equilátero**, **isósceles** ou **escaleno**. Caso contrário, imprima "Não forma triângulo".

Q3 CALCULADORA DE IMC

FÁCIL

Capítulo 2. Implemente o exemplo do capítulo 2.3 como programa completo: peça peso (kg) e altura (m), calcule o IMC e classifique. Adicione uma categoria extra "Abaixo do peso grave" para $IMC < 17$.

Q4 ANO BISSEXTO

MÉDIO

Capítulo 2. Um ano é bissexto se for divisível por 4, **exceto** se for divisível por 100 — a menos que também seja divisível por 400. Escreva um programa que verifique. Teste com: 2000 (bis), 1900 (não), 2024 (bis), 2023 (não).

Q5 CALCULADORA COM MATCH-CASE

MÉDIO

Capítulo 3. Peça ao usuário uma operação (`+` `-` `*` `/`) e dois números. Use `match-case` para selecionar a operação. Trate divisão por zero (imprima "Erro: divisão por zero") e operação inválida (case `_`).

Q6 MENU INTERATIVO COM MATCH

MÉDIO

Capítulo 3. Construa um menu: `1) Novo jogo 2) Carregar 3) Opções 4) Sair` . Use `match-case` para tratar cada opção. Para "Sair", imprima mensagem de despedida. Para qualquer outra, imprima "Função X em construção".

Q7 PRIMEIRO NÚMERO PRIMO

DIFÍCIL

Capítulo 4. Dada uma lista de números, encontre e imprima o **primeiro número primo** da lista. Use `for` + `if` + `break` . Dica: crie uma função auxiliar `eh_primo(n)` que verifica se um número é primo. Pare assim que encontrar o primeiro.

Q8 SENHA COM TENTATIVAS RESTANTES

DIFÍCIL

Capítulo 4. Modifique o exemplo do capítulo 4.4 para mostrar, a cada iteração, quantas tentativas restam. Use `while` + `if` + `break` + `continue` . Entradas vazias não contam como tentativa. Após 3 tentativas erradas (não vazias), bloqueie a conta.

Bons estudos! Pratique o código.

A programação se aprende escrevendo código. Leia o handout, execute os exemplos no Python, modifique-os, quebre-os e conserte-os. Só então resolva os exercícios. O próximo módulo será sobre **estruturas de repetição** — onde `break` e `continue` serão usados intensivamente.

01

Fundamentos

Tomada de decisão, operadores relacionais e lógicos, fluxogramas.

03

Switch & Match

Histórico em outras linguagens e o pattern matching do Python 3.10+.

02

if · elif · else

Condicional simples, binária e em cadeia. Sintaxe, exemplos e pegadinhas.

04

break & continue

Controle fino de loops: interromper ou pular iterações com responsabilidade.